

Pruning of Deep Neural Networks for Real-Time Execution on Edge TPU

Mouad Zouhdi^{1,5} Jiahe Guo^{1,6} Rafik Hammadi² Binqi Sun⁴ Philippe Leleux^{1,2,3}
Tomasz Kloda^{1,2,3} Marco Caccamo⁴

¹LAAS-CNRS, France ²Université de Toulouse, France ³Insa de Toulouse, France

⁴Technical University of Munich, Germany ⁵Sorbonne Université, France ⁶Eurocom, France

Email: mouad.zouhdi@etu.sorbonne-universite.fr, jiahe.guo@eurocom.fr, rafik.hammadi@utoulouse.fr,
{binqi.sun, mcaccamo}@tum.de, {pleleux, tkloda}@laas.fr

Abstract—Deep neural networks (DNNs) are increasingly being deployed on edge hardware accelerators in applications with stringent timing constraints. Structured pruning is well-suited for such accelerators, as it reduces the size of dense models by removing less critical components. While this technique can reduce the size of DNN models and achieve significant speedups, it also leads to some loss in the model’s original accuracy.

In this paper, we characterize the trade-off between latency and accuracy in structured pruning on a popular commercial off-the-shelf DNN accelerator, Google’s Edge Tensor Processing Unit (TPU). We present a measurement study covering several state-of-the-art baseline architectures and a diverse set of structured pruning techniques. Our results show that latency decreases sharply when pruning reduces the model size below the on-chip memory capacity, thereby eliminating the need for weight streaming from external memory. The importance criterion that best preserves accuracy is not necessarily the one that yields the greatest TPU speedup, making criterion selection an explicit accuracy–latency trade-off with no single criterion dominating both dimensions. Finally, we address DNN deployment in real-time systems by proposing a branch-and-bound method to maximize accuracy through the selection of pruning ratios for periodically executed DNNs scheduled under a non-preemptive Earliest Deadline First (EDF) policy upon a single TPU.

Index Terms—Deep Neural Networks, structured pruning, Tensor Processing Unit, real-time systems, Earliest Deadline First

I. INTRODUCTION

A. Real-Time DNN Inference on Edge Accelerators

Deep neural networks (DNNs) are increasingly deployed at the edge of cyber-physical systems, where inference often must complete within strict timing bounds. Perception in autonomous systems and closed-loop control in robotics typically impose hard or firm deadlines on each inference job. To meet these timing constraints, AI workloads are shifting from cloud-based inference, which may suffer from network delays, toward the on-device end of the edge-cloud continuum, where dedicated hardware accelerators run inference locally [1].

The Google’s Coral Edge Tensor Processing Unit (TPU) is a representative example: it runs INT8-quantized DNNs at a few Watts and is available in several embedded form factors [2]. To measure its performance and better understand its key characteristics, we compare the inference times of several DNNs executed on the Edge TPU and a popular embedded multiprocessor board.

TABLE I

UNPRUNED BASELINE INFERENCE LATENCY ON CPU AND TPU, AND MODEL SIZE SPLIT BETWEEN ON-CHIP SRAM AND OFF-CHIP MEMORY.

Model	CPU (ms)	TPU (ms)	On-chip (MB)	Off-chip (MB)	Speedup
GoogLeNet	63.0	2.0	5.66	0.00	30.9×
WRN-28-10	681.5	92.1	8.00	27.13	7.4×
ResNet-18	74.4	11.4	8.00	2.84	6.5×
MobileNetV2	9.7	1.2	2.70	0.00	7.8×
SqueezeNet 1.1	4.6	0.7	0.83	0.00	6.9×
ResNet-50	188.3	50.9	8.00	15.30	3.7×
VGG-19	70.7	39.3	8.00	11.30	1.8×

TABLE II

EDGE TPU INT8 COLD-START LATENCY FOR UNPRUNED BASELINES: FIRST VS. SECOND INFERENCE WITHIN A FRESH INTERPRETER.

Model	1st inf. (ms)	2nd inf. (ms)	Speedup
GoogLeNet	21.43	2.13	10.04×
WRN-28-10	116.66	91.13	1.28×
ResNet-18	36.94	11.51	3.21×
MobileNetV2	10.97	1.34	8.18×
SqueezeNet 1.1	3.88	0.75	5.15×
ResNet-50	75.62	50.23	1.51×
VGG-19	63.92	38.79	1.65×

B. Embedded TPU vs. Commercial Off-The-Shelf CPU

Tab. I compares the inference latency of the seven baseline architectures executed on a Raspberry Pi 4 (ARM Cortex-A72, hereafter *CPU*) and on a Coral USB Accelerator. Latency is the mean over a fixed number of timed forward passes after a warm-up phase. The measurement protocol is detailed in Section IV-B. For each model we also report the fraction of weights that reside on-chip (within the TPU’s 8 MB SRAM) and the fraction that must be streamed from host memory at each inference. We observe significant speedups (from 1.8× to 30.9×) for inferences executed on Edge TPU. The best speedup (×30.9) is achieved for *GoogLeNet* which fits entirely within the internal 8 MB SRAM of the TPU.

Tab. II compares the first inference time, during which the weights of the model are fetched into the internal SRAM, and the times of the subsequent inferences. It stands out that the memory transfers can severely impact the overall latency, with the first inference running between 1.28× (WRN-28-10) and 10.04× (GoogLeNet) slower than steady-state.

Both experiments show that large models incur high memory overhead. To address this issue, this paper investigates the application of DNN pruning techniques to reduce model size and consequently improve Edge TPU timing performance.

C. Structured Pruning of DNNs

A large body of literature has developed over the years on producing smaller models using novel *pruning* techniques [3]–[5]. However, the majority of these works have so far focused on *unstructured pruning*, which seeks to produce sparse models by zeroing out weights. These models then require software kernels, such as *XNNPACK*, or specialized hardware accelerators [6], [7] to exploit the produced sparsity in order to get any tangible acceleration benefits on conventional hardware. Furthermore, the storage gains are also only realized after applying a suitable compression scheme [8] on the sparse model weights. *Structured pruning*, on the other hand, works by removing groups of structured parameters (e.g., kernels, feature map channels, neurons, attention blocks) from the network. This has the effect of physically altering the network, and results in a slimmer dense model. The acceleration produced by structured pruning is immediately available for commercial-off-the-shelf standard hardware. This makes structured pruning the method of choice for accelerating DNN models on embedded conventional hardware platforms like *GPUs* and *TPUs*. Despite this, and until recently [9], structured pruning has been understudied in the literature.

This gap is partly due to the complexities involved in implementing structural pruning methods by hand, and which common Deep Learning libraries lack official support for. This, combined with the ubiquity of capable hardware, diminished the necessity for investigating these methods. Recently, however, there has been increased interest in model slimming, often due to embedded systems considerations, where the performance of readily available models is needed at the edge for more complex applications under resource constraints.

D. Contributions

This paper makes the following new contributions. We provide a systematic characterization of structured pruning on the Coral Edge TPU, covering seven baseline architectures, seven importance criteria, and a range of pruning ratios on CIFAR-100, reporting both classification accuracy and on-device inference time. We demonstrate an inherent accuracy-model reduction tradeoff and establish a per-model Pareto-front for different pruning methods. Finally, we propose a branch-and-bound method for the selection of pruning ratios to maximize accuracy for periodic execution of DNNs under *Earliest Deadline First (EDF)* scheduling on a single TPU.

II. RELATED WORK

Several works [10], [11] have already evaluated the performance of object classification workloads on Edge TPU and popular embedded boards. More detailed studies [12], [13] showed that memory transfers (*i.e.*, weight streaming) dominate Edge TPU inference latency.

To improve TPU performance, prior work has proposed various optimization techniques. Chen and Hong [14] introduced a model rewriting tool for replacing unsupported TPU operations with supported ones. On multi-TPUs, the *SAPar* framework [15] partitions DNN segments across different TPUs to reduce latency. On a single TPU, the *SPET* framework [16] allocates SRAM to different DNN inference tasks to meet their timing requirements and introduces the preemption points to reduce the priority inversion. Sun et al. [17], [18] studied global and partitioned real-time scheduling policies for DNN workloads on multi-TPUs.

To the best of our knowledge, the application of pruning techniques for optimizing DNN timing inference performance on Edge TPU has not yet been explored.

III. BACKGROUND: STRUCTURED PRUNING

Structured pruning (see Section I-C) entails removing grouped or structured parameter groups from the DNNs, with the added constraint of also propagating these changes to other dependent parameters. The question is how do we determine which parameters are redundant, and how many at once?

The standard approach is to rank these parameters according to some importance criteria, and prune the bottom ranking $\rho \in [0, 1)$ parameters from the network, where ρ is a parameter to be decided at pruning time, and represents the desired compression ratio. Various importance criteria have been detailed in the pruning literature, which propose different methods of ranking the parameters' importance in the network. We can make the distinction between two classes of importance criteria: *data-free* and *data-driven*. Data-free criteria do not need any other information that is not already in the weights to determine weight importance, and pruning based on this criteria does not require any further considerations than the model itself. Among these, we include the following methods:

- *Ln Magnitude* [19]: The ℓ_1/ℓ_2 magnitude of the structured parameters. This measure uses the sum of the individual weight magnitudes (or their squares) as a measure of its importance. *Group Norm* [20] applies this approach to all dependency groups across different layers.
- *Field Pruning via Geometric Median (FPGM)* [21]: A statistical measure of parameter importance, estimated through the geometric median. Parameters closer to the median are more likely to be redundant and thus pruned.

Data-driven methods, on the other hand, require access to the training data and estimate parameter importance on the basis of the parameter's salience to the output. We consider the following methods:

- *BatchNorm-Scale* [22]: This criterion uses the scaling factors γ of Batch Normalization layers as a proxy for channel importance, pruning channels whose γ approaches zero. Unlike the other data-driven criteria, its data dependency arises from a preliminary *sparsity-learning* phase in which an ℓ_1 penalty on γ is added during training, rather than from gradient-based sensitivity estimation. This criterion is not applicable to architectures without Batch Normalization (e.g., SqueezeNet 1.1).

- *Taylor* [23]: A first-order Taylor approximation of the output difference produced by removing each parameter, sometimes called sensitivity. This approximates the parameter’s contribution to the overall loss. Practically, this comes down to using the averaged gradients from a number of inputs to estimate the importance.
- *OBD-C* [24]: Pruning is based on a parameter’s second-order sensitivity to the loss, *i.e.*, using elements of the *Hessian*. Computing the full Hessian is prohibitively expensive; OBD-C approximates it via a block-diagonal Fisher information matrix (Kronecker-factored per layer), which is more accurate than the classical diagonal approximation of OBD [25]. Importance scores are obtained by a forward and backward pass over a batch of training data, using labels sampled from the model’s output distribution to form the empirical Fisher.

The difficulty of implementing structured pruning schemes lies in the fact that any grouped or structured parameter from the network has both upstream and downstream dependencies in the network, and so tracking dependencies of each removed parameter is necessary in order to modify the parameter’s dependencies and produce a valid model. For example, in a standard linear *CNN* architecture, removing a couple of weight kernels from a layer introduces the requirement of removing the corresponding input channels from the next layer. In order to get around these complexities, we rely on the recently published *Torch-Pruning* library for *ad hoc* structural pruning. This library relies on a novel *Dependency Graph* algorithm [20] that automatically tracks the dependencies of any grouped structure, and provides iterable structures for exploring dependencies in the network, while also providing the ability to alter the parameters and propagate these modifications to the dependencies.

Finally, the pruning process can be run *iteratively*, interleaving a fine-tuning step after each small pruning increment, or *one-shot*, pruning the target percentage at once followed by a single *fine-tuning* step. We adopt the one-shot variant, which is far less computationally intensive.

Our TPU benchmarking approach follows the *PruningBench* [26] testing importance criteria under a uniform recipe using a hardware-agnostic floating-point operation (FLOP) reduction proxy. We reuse its recipe and seven criteria but re-target pruning to remaining parameters and measure on-device Edge TPU latency, extending it from a software accuracy benchmark toward an accuracy–latency characterization on a real accelerator. Also, we use *DepGraph* [20] to automate the dependency analysis required when channel-pruning networks with coupled layers.

IV. TPU PRUNING EXPERIMENTS

A. Experimental Setup

The target platform is a USB Edge TPU (Coral USB Accelerator) attached to a Raspberry Pi 4 Model B (ARM Cortex-A72) running Raspberry Pi OS Lite.

The full deployment chain is depicted in Fig. 1 and comprises the following stages: training in *PyTorch*, structured

pruning with recovery fine-tuning, export to *ONNX*, conversion to *TFLite* float32, INT8 post-training quantization, and Edge TPU compilation. The structured-pruning step relies on *torch_pruning*, an established library for dependency-aware structured pruning that is only available within the *PyTorch* ecosystem. The Edge TPU, on the other hand, accepts only *TFLite* models produced by *TensorFlow*. The two frameworks describe computational graphs differently and do not support exactly the same set of operations, which makes the conversion non-trivial. Our pipeline therefore uses *ONNX* as an intermediate format: the pruned *PyTorch* model is exported to *ONNX*, converted to *TensorFlow* via *onnx2tf*, quantized with *ai-edge-quantizer* (calibrated on 100 images sampled from the CIFAR-100 training set with a fixed seed, normalized with the dataset statistics), and finally compiled for the Edge TPU.

Because structured pruning physically removes channels rather than masking them, each pruned model is a smaller dense network, and its reduced per-layer channel counts are preserved through the *ONNX* and *TFLite* conversions and through INT8 quantization, which alters only the numerical representation of the weights.

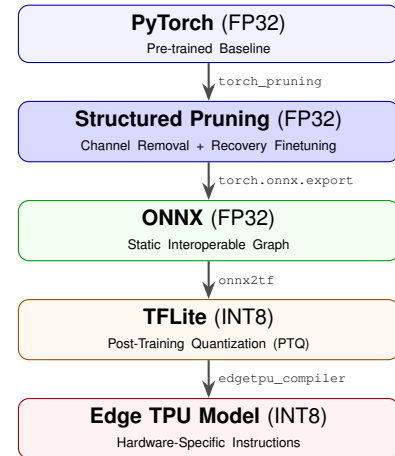


Fig. 1. End-to-end deployment workflow: from trained *PyTorch* model through structured pruning and INT8 quantization to Edge TPU execution.

We follow the *PruningBench* protocol [26] for our *CIFAR-100* classification experiments. The uniform recipe isolates the importance criterion as the only variable and gives our results a shared frame of reference with the *PruningBench* leaderboards. Our main deviation from *PruningBench* is the choice of pruning target. While *PruningBench* uses the FLOP count as its control variable to report predefined speedup ratios, we use the percentage of remaining parameters. Our parameter-centric approach is motivated by hardware constraints: Edge TPU latency is heavily bottlenecked by the model’s memory footprint. The total parameter count dictates whether the model weights can successfully fit within the 8 MB on-chip SRAM. Numerical tables are therefore not directly comparable to *PruningBench*, and individual rankings of methods may shift between the two target formulations because some criteria preferentially prune parameters with

high per-parameter compute cost. The qualitative trends should nevertheless transfer.

For every (architecture, criterion, target) triple we start from the same trained baseline and prune progressively over 400 incremental steps under global ranking, retaining at least 10% of the channels of every prunable layer, which is a threshold set to limit overpruning of layers, and no fine-tuning is performed between steps. The pruning target is expressed as a reduction in remaining parameters and is swept over $\{10, 20, \dots, 90\}\%$. The realized reduction can fall slightly short of the target when the per-layer protection saturates, and we report the realized value throughout. The data-driven criteria (Taylor and OBD-C) estimate importance from ten minibatches before each step, OBD-C using labels sampled from the model’s output distribution to form the empirical Fisher information.

Once the target is reached, every model undergoes a single recovery fine-tuning of 100 epochs, rather than fine-tuning at each step. We use SGD (learning rate 0.01, momentum 0.9, weight decay 5×10^{-4} , batch size 128), with the learning rate decayed by 0.1 at epochs 60 and 80, and the standard CIFAR-100 augmentation (random crop with four-pixel padding and horizontal flip). No dropout or other explicit regularization is used beyond weight decay and this augmentation. The BatchNorm-scale criterion additionally requires a preliminary sparsity-learning phase, in which an ℓ_1 penalty on the BatchNorm scales (coefficient 10^{-5}) is applied during a 100-epoch run (same optimizer and schedule, with weight decay disabled) so that the scales become separable; this phase is inapplicable to SqueezeNet 1.1, which has no BatchNorm layers.

Seven CNN architectures are chosen to cover a representative range of architectures deployable on Edge TPU:

- *ResNet-18* (11 M parameters) and *ResNet-50* (24 M parameters) [27]. Architectures with residual connections, widely used in image classification.
- *VGG-19* (20 M parameters) [28]. A monolithic architecture without skip connections, representative of older designs dominated by stacked convolutional layers.
- *WRN-28-10* (37 M parameters) [29]. A *Wide Residual Network*, a widened version of the *ResNets*, chosen to test a strongly memory-constrained regime.
- *MobileNetV2* (2.4 M parameters) [30]. An architecture designed for mobile use through depthwise separable convolutions, representative of compact designs.
- *GoogLeNet* (5.7 M parameters) [31]. An architecture with parallel branches (Inception modules), representative of non-linear topologies.
- *SqueezeNet 1.1* (0.8 M parameters) [32]. A very compact architecture, chosen as a limiting case for very small models without *BatchNorm*.

B. Measurement Methodology

Latency is measured on the target hardware (Raspberry Pi 4 for CPU, Coral USB Accelerator for TPU) using the TFLite interpreter in INT8 mode. For each model and device, the latency benchmark performs a full forward pass of the model on a fixed synthetic input tensor (drawn once with a fixed

random seed and reused throughout); since INT8 latency is data-independent by construction, the seed removes run-to-run variance without biasing the estimate. Each measurement begins with 30 warm-up forward passes whose timings are discarded, followed by 200 timed passes; the warm-up size follows from steady state being reached within 10–15 invocations on Edge TPU, and 200 samples suffice for the coefficient of variation to stabilise below 1% for on-chip models and 3% for streaming-dominated ones, with no change beyond that. We report the mean over the timed passes. Because each measurement times the complete interpreter invocation, any weight streaming from host memory is included in the reported latency, which is precisely the effect captured by the 8 MB on-chip threshold discussed in Section IV-C. The same protocol is applied to every model so that results are directly comparable, and CPU and TPU latencies are measured independently in their respective execution environments.

Top-1 and top-5 accuracy are evaluated on the full CIFAR-100 test set. For the FP32 reference accuracy, inference is performed on the original PyTorch model on a GPU, before TFLite conversion. This avoids the potential numerical drift introduced by the PyTorch $\rightarrow$ ONNX $\rightarrow$ TFLite chain and reduces evaluation time. For INT8 models, accuracy is evaluated on the Edge TPU, with the Raspberry Pi used only as host: each image is preprocessed on the Pi, dispatched to the Coral USB Accelerator via the Edge TPU runtime, and the TPU output is read back to compute the top-1 and top-5 statistics.

C. Discussion

1) *Latency analysis:* Before turning to the pruned models, we first examine the unpruned baselines on CPU and TPU. Prior studies of the Edge TPU have shown that its performance degrades sharply once the model size exceeds the on-chip SRAM (around 8 MB): the weights that do not fit on-chip must be streamed from external memory at each inference, and these transfers dominate the latency [12], [13]. We observe this effect directly in our panel. GoogLeNet, whose weights fit entirely on-chip, attains the highest TPU/CPU speedup, while VGG-19 attains the lowest (Tab. I), consistent with a large fraction of its weights being streamed from host memory at each inference.

For pruned models, the effect is clear: for every model whose unpruned footprint exceeds 8 MB, the TPU speedup increases sharply as the pruned footprint approaches and then crosses the 8 MB threshold, at which point weight streaming disappears. The magnitude of this jump, and the spread between importance criteria around the crossing, varies across models, but the threshold behavior itself is consistent throughout the panel (Fig. 2).

2) *Accuracy analysis:* Top-1 accuracy decreases only modestly with compression across all seven architectures (Fig. 3). A clear gap appears only at the highest pruning rates. At low and moderate rates, the random criterion remains close to the informed criteria across the panel. This is consistent with the observation that, given a sufficiently generous recovery fine-tuning budget, the differences between importance criteria tend

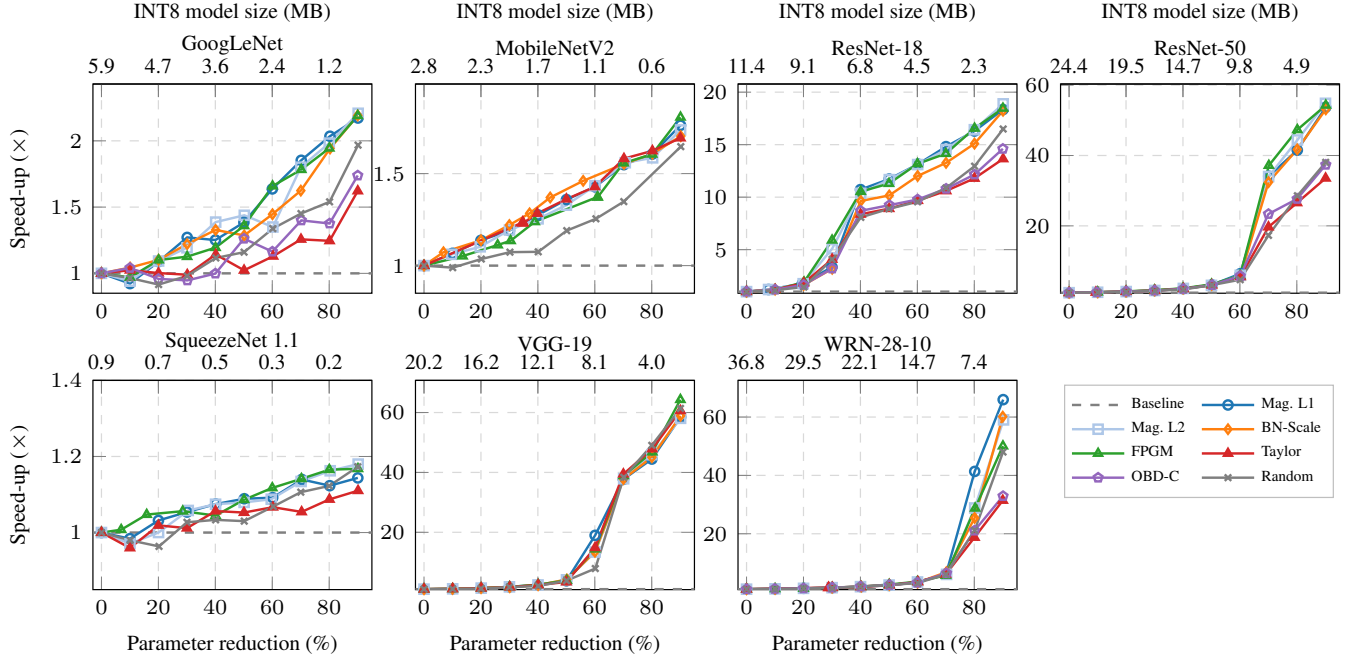


Fig. 2. Edge TPU speed-up ($\times$) versus real parameter reduction for each pruning criterion across seven CIFAR-100 architectures. Dashed lines mark the unpruned baseline (speed-up = $1\times$).

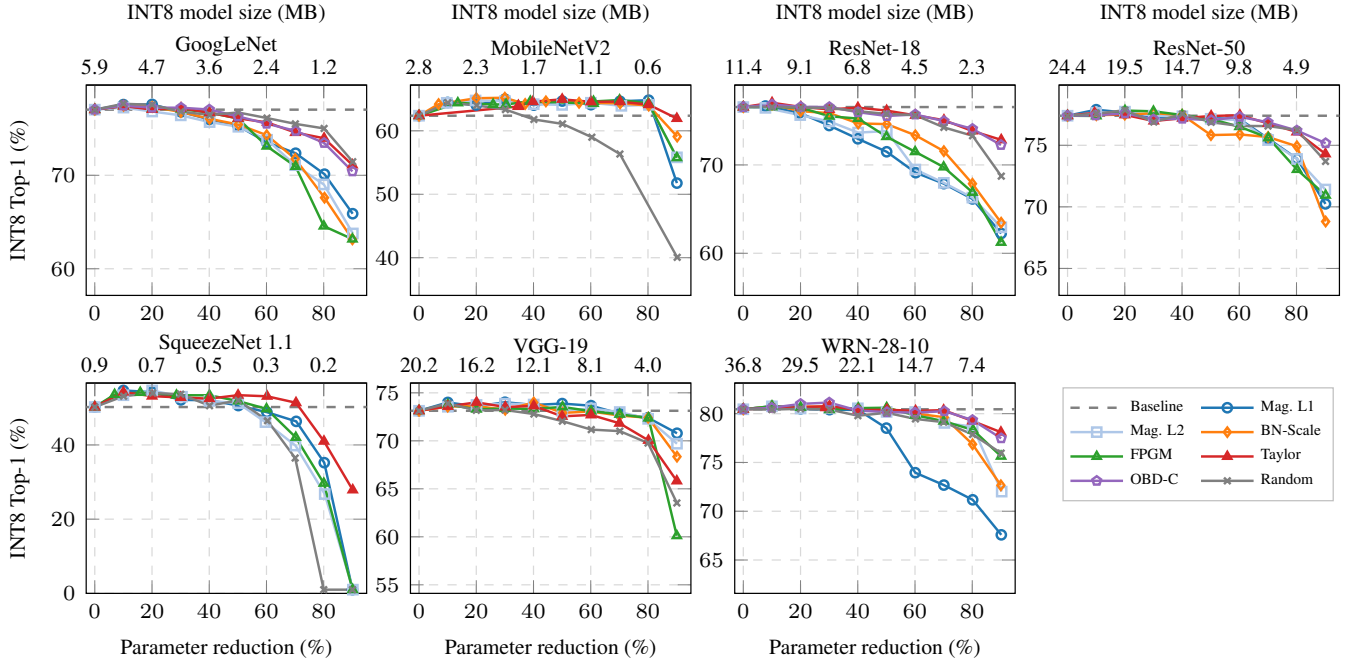


Fig. 3. Top-1 INT8 accuracy versus real parameter reduction for each pruning criterion across seven CIFAR-100 architectures on the Coral Edge TPU. Dashed lines mark the unpruned baseline.

to wash out after fine-tuning [33], as does the PruningBench recipe used here. Because post-recovery accuracy separates the criteria only at the highest pruning rates, we also examine two finer findings: the accuracy a criterion retains before fine-

tuning, and the speed at which accuracy recovers during it.

The first finding is the accuracy a criterion preserves immediately after pruning, before any recovery fine-tuning (Fig. 4). The criteria diverge sharply in this respect on ResNet-50

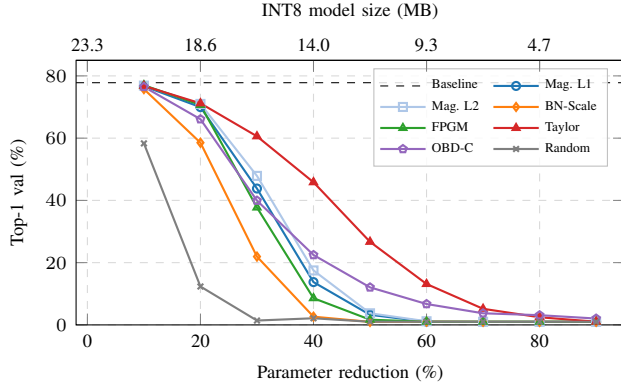


Fig. 4. ResNet-50 Top-1 validation accuracy immediately after pruning, before any recovery fine-tuning.

(Tab. A): at a 40% reduction Taylor still retains 45.8% top-1 with no fine-tuning and OBD-C 22.5%, whereas magnitude- ℓ_1 has fallen to 13.8% and random to chance (2.1% on CIFAR-100). The data-driven criteria therefore preserve far more accuracy before recovery; the magnitude-based criteria and random reach a comparable final accuracy only because the fine-tuning budget compensates for what they discard at pruning time.

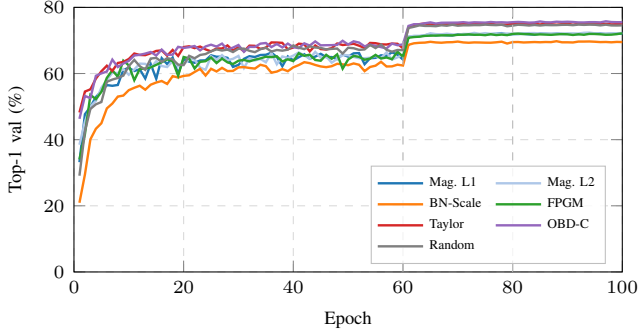


Fig. 5. ResNet-50 recovery fine-tuning trajectory at a 90% parameter reduction. Vertical dashed lines mark the learning-rate decays at epochs 60 and 80.

The second finding is the speed of accuracy recovery during fine-tuning. At moderate compression rates the recovery trajectories are similar across criteria, with only a small drop before recovery; at the highest rates they separate (Fig. 5, 90% reduction), where the data-driven criteria recover to a markedly higher accuracy than the magnitude-based ones.

Finally, the criterion that best preserves accuracy is not the one that runs fastest on the TPU. On ResNet-50 this decorrelation emerges at the high reduction targets where the recovered accuracies finally separate. Beyond a 60% reduction the pruned weights fit entirely on-chip, so the remaining latency differences between criteria reflect only which channels each criterion removes. In this regime the data-driven criteria (Taylor and OBD-C) retain the most accuracy (Fig. 3) but yield the smallest TPU speedups (Fig. 2), whereas the magnitude-

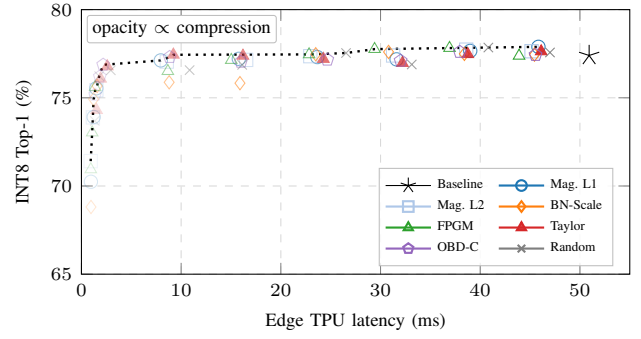


Fig. 6. Accuracy-latency Pareto front for pruned ResNet-50. The star marks the unpruned baseline; marker opacity is proportional to the compression ratio.

based criteria and FPGM achieve the largest speedups at a clear accuracy cost (Spearman $\rho \approx -0.8$ at an 80% reduction, $p < 0.05$). At a 90% reduction, for instance, OBD-C retains 75.2% top-1 (95% CI: [74.3, 76.1]) at a $37\times$ TPU speedup, whereas magnitude- ℓ_2 reaches $55\times$ at only 71.4% (95% CI: [70.5, 72.3]); the two intervals do not overlap. This accuracy-latency relationship is visible as the Pareto front in Fig. 6. The structures that are most redundant for accuracy are therefore not those whose removal yields the largest latency gain, which makes the choice of importance criterion an explicit accuracy-latency trade-off.

V. PRUNING RATE SELECTION FOR REAL-TIME SYSTEMS

Given a set of DNNs with different execution rates (*i.e.*, periods) and different pruning ratios, each resulting in a specific accuracy and inference time, we formulate an optimization problem for selecting the pruning ratio for each DNN to maximize the overall system accuracy and guarantee a feasible execution on a single TPU (*i.e.*, no deadline misses). We solve the pruning rate assignment problem by applying a *branch-and-bound (B&B)* method.

A. System Model

We consider a task set τ consisting of n sporadic DNN tasks. Each DNN task $\tau_i \in \tau$ has a minimum inter-arrival time T_i between releases of its jobs, and must finish before its deadline T_i time units after its release (*i.e.*, implicit deadlines). We say that the task is *schedulable* if all its jobs can be completed before their deadlines. Task τ_i worst-case execution time C_i depends on its pruning rate and can be selected among at most m different values. For each pruning rate $1 \leq j \leq m$, task τ_i worst-case execution time is given by C_i^j and its accuracy by Acc_i^j . We assume that task τ_i 's higher pruning rate has lower accuracy and faster worst-case inference time: $\forall 1 \leq j < m : C_i^j > C_i^{j+1}$ and $Acc_i^j > Acc_i^{j+1}$. The utilization of a task is defined as $U_i = C_i/T_i$, and the utilization of the task set is calculated as $U = \sum_{i=1}^n U_i$.

We consider that the DNN tasks are executing on a single TPU. To avoid overheads due to weight streaming (see Tab. II), we assume that the tasks are scheduled by non-preemptive *Earliest Deadline First (EDF)* (*i.e.*, every job executes from its

start uninterruptedly until completion and at every idle instant a job with the earliest absolute deadline among all the ready jobs is selected for execution).

B. Non-Preemptive EDF Schedulability Conditions

We recap the schedulability conditions for a non-preemptive *Earliest Deadline First (EDF)* upon a single processing unit [34], [35]. The analysis is based on the concept of *demand bound function* ($dbf(\Delta)$), which quantifies the workload that must be executed within any time interval of length $\Delta > 0$:

$$dbf(\Delta) = \sum_{i=1}^n \left\lfloor \frac{\Delta}{T_i} \right\rfloor \cdot C_i \quad (1)$$

In the non-preemptive *EDF*, a priority inversion might happen when a job is released and must wait for the completion of a currently executing job with a larger deadline (lower-priority). As dbf accounts for the interference of the jobs that are released and must be completed within the interval Δ , a blocking from a job with the deadline exceeding the length of the interval Δ is taken into account by the *blocking* function:

$$b(\Delta) = \max\{C_i \mid T_i > \Delta, \tau_i \in \tau\} \quad (2)$$

The following condition ensures that there is no interval of length Δ during which the workload that must be completed exceeds the total available processor time within this interval:

$$\forall \Delta > 0 : b(\Delta) + dbf(\Delta) \leq \Delta \quad (3)$$

As the $dbf(\Delta)$ can be upper-bounded by $U \cdot \Delta$, Jeffay et al. [36] proposed the following polynomial-time test (sufficient condition) assuming that tasks are sorted in non-decreasing order of their periods and introducing $B_k = b(T_k)$:

$$B_k/T_k + \sum_{i=1}^k U_i \leq 1, \quad \forall 1 \leq k \leq n \quad (4)$$

C. Real-Time Tasks Pruning Rate Optimization

Our goal is to maximize overall task set accuracy by determining the pruning ratio for each DNN task while preserving task set schedulability. We develop a 0–1 integer linear programming (*ILP*) model and propose a *Branch-and-Bound (B&B)* method to solve it.

$$\max. \quad \sum_{i=1}^n \sum_{j=1}^m \frac{Acc_i^j}{T_i} \cdot x_i^j \quad (5)$$

$$\text{s.t.} \quad B_k/T_k + \sum_{i=1}^k \sum_{j=1}^m x_i^j \cdot C_i^j/T_i \leq 1, \quad \forall 1 \leq k \leq n \quad (6)$$

$$B_k \geq C_i^j \cdot x_i^j, \quad \forall k < i \leq n, \quad \forall 1 \leq j \leq m \quad (7)$$

$$\sum_{j=1}^m x_i^j = 1, \quad \forall 1 \leq i \leq n \quad (8)$$

$$x_i^j \in \{0, 1\}, \quad \forall 1 \leq i \leq n, \quad 1 \leq j \leq m \quad (9)$$

Objective (5) maximizes the task set accuracy. Constraint (6) guarantees the non-preemptive *EDF* schedulability given by

Condition (4). Constraint (7) computes the blocking function from Equation (2) for each task. Constraint (8) ensures that each task runs only one version with a given pruning rate. Binary decision variables are introduced with Constraint (9) (e.g., if $x_i^j = 1$, task τ_i always runs its j -th pruning version).

We propose a *branch-and-bound (B&B)* algorithm to determine the task pruning rates that maximize the overall task set accuracy. At each branching step, the algorithm selects a pruning rate for one task and generates child branches by considering all feasible pruning rates for the next task. When creating a new branch, it verifies whether the selected pruning rate satisfies Condition (4) for the schedulability of all previously assigned tasks. To reduce the search space, we employ two pruning strategies. The first strategy discards partial solutions that remain unschedulable even under the most aggressive pruning assumption, where all remaining tasks are assigned their highest pruning rates and therefore have the shortest worst-case execution times. The second strategy eliminates partial solutions whose maximum achievable accuracy, assuming that all remaining tasks attain their highest possible accuracies, cannot exceed the best accuracy found so far.

The proposed method is detailed in Alg. 1. Its main component is the `max_accuracy` function. It takes the following inputs: a task set τ sorted in non-decreasing order of periods, a pruning rate assignment $v^{(i-1)}$ for tasks $\tau_1, \tau_2, \dots, \tau_{i-1}$ (e.g., $v^{(3)} = \{1, 3, 5\}$ means that tasks τ_1, τ_2 and τ_3 take respectively C_1^1, C_2^3 and C_3^5 time units to execute and their accuracies are Acc_1^1, Acc_2^3 and Acc_3^5), the best solution found so far with pruning rate assignment v^* for all tasks in τ and its accuracy a^* , and the maximum blocking B from tasks $\tau_i, \dots, \tau_n$ that tasks $\tau_1, \dots, \tau_{i-1}$ can tolerate under the pruning rate assignment $v^{(i-1)}$. The function returns the best pruning rate assignment v^* found so far together with its accuracy a^* (calculated as the objective (5) by a helper function `acc`).

Before the search begins, Alg. 1 initializes the best solution v^* to the assignment in which all tasks are assigned their maximum pruning rates (Line 21). If this assignment is not schedulable, no feasible solution exists and the algorithm terminates (Line 26). Otherwise, the search is initiated in Line 24. The `max_accuracy` function first checks whether the partial assignment $v^{(i-1)}$ specifies pruning rates for all n tasks (Line 2). If so, its schedulability and accuracy are evaluated (Line 3). If the assignment is schedulable and achieves a higher accuracy than the current best solution a^* , it becomes the new best solution and is returned (Line 4). If $v^{(i-1)}$ is not yet a complete assignment, the algorithm applies two branch-pruning techniques. The first one, in Lines 6–8, constructs a complete assignment v_{util}^{min} by assigning the maximum pruning rates to all remaining tasks. If this assignment is not schedulable, the current branch is discarded. Indeed, decreasing pruning rates can only increase task execution times and utilization, and therefore cannot restore schedulability. The second pruning technique, in Lines 9–13, constructs a complete assignment v_{acc}^{max} by assigning the highest-accuracy pruning rates to all remaining tasks. If the resulting upper bound on the achievable accuracy does not exceed the accuracy

Algorithm 1: Accuracy maximization using B&B

Input: Set of n tasks $\tau = \{\tau_1, \dots, \tau_n\}$ sorted by periods, each with no more than m pruned versions

Output: Task versions $v^* = \{v_1, \dots, v_n\}$ with the maximal task set accuracy a^*

```

1 Function max_accuracy( $\tau, v^{(i-1)}, v^*, a^*, B$ ):
2   if  $i - 1 = n$  then
3     if  $\tau$  is schedulable with  $v^{(n)}$  and  $\text{acc}(v^{(n)}) > a^*$ 
4       then
5          $v^* \leftarrow v^{(n)}$ ;  $a^* \leftarrow \text{acc}(v^{(n)})$ ;
6       return  $v^*, a^*$ ;
7    $v_{\text{util}}^{\min} \leftarrow \{v_1, \dots, v_{i-1}, v_i = m, \dots, v_n = m\}$ ;
8   if  $\tau$  is not schedulable with  $v_{\text{util}}^{\min}$  then
9     return  $v^*, a^*$ ;
10   $v_{\text{acc}}^{\max} \leftarrow \{v_1, \dots, v_{i-1}, v_i = 1, \dots, v_n = 1\}$ ;
11  if  $\text{acc}(v_{\text{acc}}^{\max}) \leq a^*$  then
12    return  $v^*, a^*$ ;
13  else if  $\tau$  is schedulable with  $v_{\text{acc}}^{\max}$  then
14    return  $v_{\text{acc}}^{\max}, \text{acc}(v_{\text{acc}}^{\max})$ ;
15   $B \leftarrow \max\{B, (1 - U(v^{(i-1)})) \cdot T_i\}$ ;
16   $v_i \leftarrow \arg \min_j \{C_i^j \leq B\}$ ;
17  while  $v_i \leq m$  do
18     $v^{(i)} \leftarrow \{v_1, \dots, v_{i-1}, v_i\}$ ;
19     $v^*, a^* \leftarrow \text{max\_accuracy}(\tau, v^{(i)}, v^*, a^*, B)$ ;
20     $v_i \leftarrow v_i + 1$ ;
21  return  $v^*, a^*$ ;
22   $v^* \leftarrow \{v_1 = 1, \dots, v_n = 1\}$ ;
23  if  $\tau$  is schedulable with  $v^*$  then
24     $a^* \leftarrow \text{acc}(v^*)$ ;  $v^{(0)} \leftarrow \emptyset$ ;  $B \leftarrow T_1$ ;
25    return max_accuracy( $\tau, v^{(0)}, v^*, a^*, B$ );
26  else
27    return  $\emptyset, 0$ ;
28  Function  $U(v)$ :
29     $U \leftarrow 0, i \leftarrow 1$ ;
30    foreach  $j \in v$  do
31       $U \leftarrow U + C_i^j / T_i$ ;
32       $i \leftarrow i + 1$ ;
33    return  $U$ ;
34  Function  $\text{acc}(v)$ :
35     $\text{Acc} \leftarrow 0, i \leftarrow 1$ ;
36    foreach  $j \in v$  do
37       $\text{Acc} \leftarrow \text{Acc} + \text{Acc}_i^j / T_i$ ;
38       $i \leftarrow i + 1$ ;
39    return  $\text{Acc}$ ;

```

of the current best solution, the branch is discarded because it cannot yield a better solution. If neither pruning condition applies, the algorithm computes in Lines 14–15 the maximum worst-case execution time that can be assigned to task τ_i while ensuring that the schedulability condition (4) remains satisfied for all $k < i$. Finally, for each pruning rate of task τ_i whose execution time does not exceed this limit, the partial assignment $v^{(i-1)}$ is extended to a new assignment $v^{(i)}$, and the search continues recursively.

D. Schedulability Experiments

Based on the obtained benchmark profiles and using *Rand-fixedsum* algorithm [37], we generate random sets of $n = 6$ tasks. For each utilization $U \in \{0.1, 0.2, \dots, 4.0\}$, we randomly generate 250 such task sets where each task has a speedup and accuracy profile selected randomly from benchmarks shown in Fig. 2 and 3. Task periods are sampled from interval $[10, 100]$ ms using a log-uniform distribution. We compare the schedulability for each task set without pruning (*baseline*) using Condition (4) and with pruning (*B&B*) by applying Alg. 1. Fig. 7 illustrates the percentage of all task sets deemed schedulable by each method.

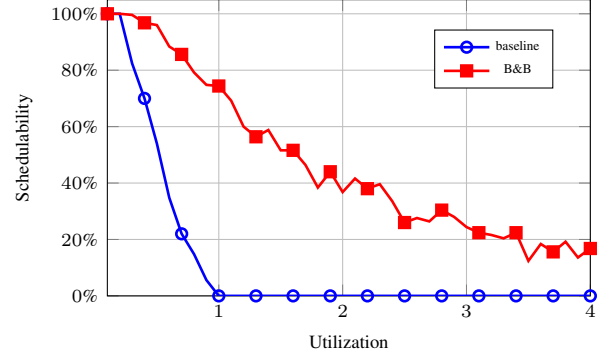


Fig. 7. Schedulability ratio with and without pruning.

Additionally, we report in Fig. 8 the accuracy loss for *B&B* method calculated with respect to the original accuracy (calculated also for non-schedulable unpruned task sets). As expected, the accuracy loss increases with utilization, reaching 2% for high-utilization ($U \geq 4$) task sets.

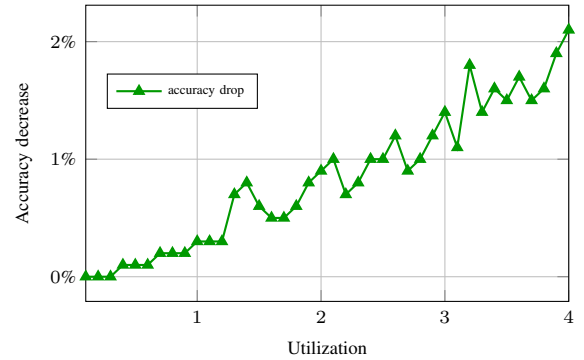


Fig. 8. Accuracy drop with pruning for B&B method.

VI. CONCLUSIONS AND FUTURE WORK

In this paper, we explored the application of structured pruning to DNN models executed on a single *TPU*. Our results demonstrate that inference time can be significantly reduced when pruning shrinks the model sufficiently to fit within the *TPU*'s on-chip memory. Moreover, model accuracy can be largely preserved even after reducing up to 60% of the parameters. For multi-rate systems executing multiple DNN

workloads, we formulated the pruning-rate selection problem as a 0-1 integer linear programming (ILP) model and proposed a Branch-and-Bound (B&B) method to determine the pruning rate of each DNN under non-preemptive *EDF* scheduling.

In future work, we plan to extend our benchmarking framework to multi-TPU systems and investigate TPU memory sharing among multiple DNN models (*co-compilation*).

ACKNOWLEDGEMENT

Marco Caccamo was supported by an Alexander von Humboldt Professorship endowed by the German Federal Ministry of Education and Research. This work was sponsored in part by ANR JCJC grant ARTi (ANR-25-CE25-7918).

REFERENCES

- [1] V. Avelar, P. Donovan, P. Lin, W. Torell, and M. A. T. Arango, "White Paper 110 The AI Disruption: Challenges and Guidance for Data Center Design," Schneider Electric – Energy Management Research Center, Tech. Rep., 2023.
- [2] Google Coral, "Products," 2026, accessed: 2026-05-26. [Online]. Available: <https://www.coral.ai/products/>
- [3] Y. Guo, A. Yao, and Y. Chen, "Dynamic Network Surgery for Efficient DNNs," 2016. [Online]. Available: <https://arxiv.org/abs/1608.04493>
- [4] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," 2015. [Online]. Available: <https://arxiv.org/abs/1506.02626>
- [5] X. Dong, S. Chen, and S. J. Pan, "Learning to prune deep neural networks via layer-wise optimal brain surgeon," 2017. [Online]. Available: <https://arxiv.org/abs/1705.07565>
- [6] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: Efficient inference engine on compressed deep neural network," 2016. [Online]. Available: <https://arxiv.org/abs/1602.01528>
- [7] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [8] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," 2016. [Online]. Available: <https://arxiv.org/abs/1510.00149>
- [9] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 5, pp. 2900–2919, 2024.
- [10] A. A. Asyraf Jainuddin, Y. C. Hou, M. Z. Baharuddin, and S. Yusoff, "Performance Analysis of Deep Neural Networks for Object Classification with Edge TPU," in *2020 8th International Conference on Information Technology and Multimedia (ICIMU)*, 2020, pp. 323–328.
- [11] S. Sam Xu and S. Shue, "Real-Time Object Detection Algorithm Performance on Edge Computing Devices," in *2024 International Symposium on Networks, Computers and Communications (ISNCC)*, 2024, pp. 1–5.
- [12] J. Villarrubia, L. Costero, F. D. Igual, and K. Olcoz, "Improving inference time in multi-TPU systems with profiled model segmentation," in *2023 31st Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2023, pp. 84–91.
- [13] H. Gao, H. Choi, and Y. Wang, "Work-In-Progress: Modeling and Analysis of Inference Latency on Usb Edge TPUs," in *2025 IEEE Real-Time Systems Symposium (RTSS)*, 2025, pp. 600–603.
- [14] K.-F. Chen and D.-Y. Hong, "Rewriting Deep Learning Models for Maximizing Edge TPU Utilization," in *2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)*, 2023, pp. 657–664.
- [15] B. Sun, B. Zou, Y. Hu, T. Kloda, L. Wang, T. Abdelzaher, and M. Caccamo, "SAPar: A Surrogate-Assisted DNN Partitioner for Efficient Inferences on Edge TPU Pipelines," *ACM Trans. Embed. Comput. Syst.*, vol. 24, no. 5s, Sep. 2025.
- [16] C. Han, H. S. Chwa, K. Lee, and S. Oh, "SPET: Transparent SRAM Allocation and Model Partitioning for Real-time DNN Tasks on Edge TPU," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–6.
- [17] B. Sun, T. Kloda, J. Chen, C. Lu, and M. Caccamo, "Schedulability analysis of non-preemptive sporadic gang tasks on hardware accelerators," in *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2023, pp. 147–160.
- [18] B. Sun, T. Kloda, C.-g. Wu, and M. Caccamo, "Partitioned Scheduling and Parallelism Assignment for Real-Time DNN Inference Tasks on Multi-TPU," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024.
- [19] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning Filters for Efficient ConvNets," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=rJqFGTslg>
- [20] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "Depgraph: Towards any structural pruning," 2023. [Online]. Available: <https://arxiv.org/abs/2301.12900>
- [21] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang, "Filter pruning via geometric median for deep convolutional neural networks acceleration," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4335–4344.
- [22] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2755–2763.
- [23] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz, "Importance estimation for neural network pruning," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 11 256–11 264.
- [24] C. Wang, R. Grosse, S. Fidler, and G. Zhang, "EigenDamage: Structured Pruning in the Kronecker-Factored Eigenbasis," 2019. [Online]. Available: <https://arxiv.org/abs/1905.05934>
- [25] Y. L. Cun, J. S. Denker, and S. A. Solla, *Optimal brain damage*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1990, p. 598–605.
- [26] C. Li, H. Li, M. Xue, G. Fang, S. Zhou, Z. Feng, H. Wang, M. Song, and J. Song, "A Comprehensive Study of Structural Pruning for Vision Models," Jan. 2025.
- [27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [28] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [29] S. Zagoruyko and N. Komodakis, "Wide residual networks," 2017. [Online]. Available: <https://arxiv.org/abs/1605.07146>
- [30] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," 2019. [Online]. Available: <https://arxiv.org/abs/1801.04381>
- [31] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [32] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," 2016. [Online]. Available: <https://arxiv.org/abs/1602.07360>
- [33] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the Value of Network Pruning," Mar. 2019. [Online]. Available: <http://arxiv.org/abs/1810.05270>
- [34] L. George, N. Rivierre, and M. Spuri, "Preemptive and Non-Preemptive Real-Time UniProcessor Scheduling," 1996. [Online]. Available: <https://inria.hal.science/inria-00073732>
- [35] S. Baruah and S. Chakraborty, "Schedulability analysis of non-preemptive recurring real-time tasks," in *IEEE International Parallel & Distributed Processing Symposium*, 2006.
- [36] K. Jeffay, D. Stanat, and C. Martel, "On non-preemptive scheduling of period and sporadic tasks," in *[1991] Proceedings Twelfth Real-Time Systems Symposium*, 1991, pp. 129–139.
- [37] P. Emberson, R. Stafford, and R. Davis, "Techniques for the synthesis of multiprocessor tasksets," in *1st International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems at the Euromicro Conference on Real-Time Systems*, Jul. 2010, pp. 6–11.

APPENDIX A
EXPERIMENTAL RESULTS FOR RESNET-50

Target	Importance	Accuracy		Time			Reduction (%)		Memory (mb)		
		Acc	Δ Acc	Lat (ms)	TPU spd	CPU spd	Params	MACs	Size	On Chip	Off Chip
–	baseline	77.40	+0.00	50.92	1.00×	1.00×	0.0	0.0	23.30	7.66	15.26
10%	magnitude_l1	77.89	+0.49	45.84	1.11×	1.32×	10.0	28.5	20.98	7.66	13.59
	magnitude_l2	77.64	+0.24	45.15	1.13×	1.31×	10.0	27.1	20.97	7.66	13.50
	bn_scale	77.45	+0.05	45.44	1.12×	1.25×	10.1	24.4	20.97	7.66	13.54
	fpgm	77.38	-0.02	43.90	1.16×	1.24×	10.0	20.2	20.96	7.65	13.02
	taylor	77.63	+0.23	46.11	1.10×	1.33×	9.4	29.8	21.12	7.65	13.75
	obdc	77.43	+0.03	45.55	1.12×	1.35×	9.8	29.5	21.03	7.66	13.61
	random	77.57	+0.17	47.00	1.08×	1.07×	10.1	10.4	20.99	7.65	14.04
20%	magnitude_l1	77.66	+0.26	39.00	1.31×	1.55×	20.0	40.5	18.65	7.66	11.57
	magnitude_l2	77.73	+0.33	38.59	1.32×	1.52×	20.1	38.7	18.64	7.66	11.48
	bn_scale	77.49	+0.09	38.41	1.33×	1.50×	20.0	37.1	18.65	7.66	11.45
	fpgm	77.82	+0.42	36.92	1.38×	1.43×	20.1	32.2	18.62	7.66	10.98
	taylor	77.47	+0.07	38.76	1.31×	1.62×	20.0	43.1	18.66	7.66	11.54
	obdc	77.60	+0.20	37.94	1.34×	1.60×	20.1	41.6	18.64	7.66	11.29
	random	77.85	+0.45	40.82	1.25×	1.18×	20.1	20.1	18.69	7.65	12.17
30%	magnitude_l1	77.18	-0.22	31.61	1.61×	1.84×	30.1	51.0	16.33	7.66	9.36
	magnitude_l2	77.40	+0.00	31.26	1.63×	1.80×	30.1	49.7	16.32	7.66	9.27
	bn_scale	77.60	+0.20	30.80	1.65×	1.79×	30.1	48.0	16.31	7.66	9.12
	fpgm	77.76	+0.36	29.40	1.73×	1.74×	30.0	45.3	16.31	7.66	8.69
	taylor	76.97	-0.43	32.20	1.58×	1.83×	30.1	51.2	16.33	7.66	9.53
	obdc	77.10	-0.30	32.01	1.59×	1.82×	30.0	50.0	16.34	7.66	9.50
	random	76.89	-0.51	33.13	1.54×	1.33×	30.1	30.4	16.38	7.66	9.73
40%	magnitude_l1	77.31	-0.09	23.66	2.15×	2.26×	40.1	61.2	14.00	7.66	6.95
	magnitude_l2	77.36	-0.04	22.96	2.22×	2.20×	40.1	59.8	13.99	7.66	6.74
	bn_scale	77.47	+0.07	23.50	2.17×	2.18×	40.1	59.1	13.99	7.66	6.89
	fpgm	77.46	+0.06	22.83	2.23×	2.14×	40.0	57.2	13.99	7.65	6.67
	taylor	77.20	-0.20	24.27	2.10×	2.06×	40.2	57.1	14.03	7.66	7.11
	obdc	77.16	-0.24	24.66	2.07×	2.05×	40.1	56.2	14.03	7.66	7.25
	random	77.54	+0.14	26.54	1.92×	1.55×	40.1	41.5	14.08	7.63	7.81
50%	magnitude_l1	77.21	-0.19	15.79	3.23×	2.79×	50.2	69.5	11.66	7.66	4.57
	magnitude_l2	77.15	-0.25	16.52	3.08×	2.75×	50.0	68.9	11.70	7.69	4.77
	bn_scale	75.83	-1.57	15.90	3.20×	2.73×	50.1	68.7	11.65	7.66	4.62
	fpgm	77.14	-0.26	15.03	3.39×	2.79×	50.0	68.3	11.66	7.66	4.34
	taylor	77.38	-0.02	16.20	3.14×	2.30×	50.2	62.3	11.74	7.66	4.63
	obdc	77.04	-0.36	16.00	3.18×	2.30×	50.2	62.1	11.71	7.66	4.51
	random	76.87	-0.53	16.12	3.16×	1.77×	50.2	50.0	11.77	7.65	4.52
60%	magnitude_l1	77.12	-0.28	7.96	6.40×	3.78×	60.2	78.2	9.34	7.73	2.03
	magnitude_l2	77.05	-0.35	8.56	5.95×	3.71×	60.1	77.8	9.35	7.70	2.25
	bn_scale	75.88	-1.52	8.80	5.79×	3.68×	60.2	77.4	9.31	7.66	2.41
	fpgm	76.51	-0.89	8.67	5.88×	3.90×	60.1	78.4	9.34	7.70	2.34
	taylor	77.44	+0.04	9.22	5.52×	2.56×	60.2	67.0	9.44	7.66	2.40
	obdc	77.31	-0.09	8.85	5.75×	2.63×	60.1	67.4	9.44	7.66	2.35
	random	76.57	-0.83	10.85	4.69×	2.16×	60.1	61.2	9.47	7.66	2.82
70%	magnitude_l1	75.54	-1.86	1.49	34.12×	5.50×	70.1	85.7	7.02	7.68	0.00
	magnitude_l2	75.43	-1.97	1.50	34.02×	5.55×	70.1	86.1	7.02	7.71	0.00
	bn_scale	75.67	-1.73	1.57	32.42×	5.35×	70.1	85.2	7.02	7.73	0.03
	fpgm	75.60	-1.80	1.37	37.16×	6.01×	70.2	87.1	6.98	7.52	0.00
	taylor	76.78	-0.62	2.59	19.66×	2.96×	70.0	72.4	7.18	7.66	0.23
	obdc	76.87	-0.53	2.18	23.37×	3.10×	70.0	73.0	7.15	7.66	0.05
	random	76.56	-0.84	2.94	17.29×	2.74×	70.0	70.6	7.17	7.66	0.43
80%	magnitude_l1	73.91	-3.49	1.23	41.52×	8.37×	80.2	91.9	4.68	5.15	0.00
	magnitude_l2	73.89	-3.51	1.15	44.41×	9.02×	80.2	92.5	4.66	5.18	0.00
	bn_scale	74.92	-2.48	1.22	41.74×	8.18×	80.1	91.5	4.68	5.26	0.00
	fpgm	73.03	-4.37	1.08	47.31×	9.49×	80.2	93.2	4.63	5.05	0.00
	taylor	76.08	-1.32	1.92	26.54×	3.69×	80.1	79.4	4.85	5.57	0.00
	obdc	76.20	-1.20	1.84	27.67×	3.89×	80.1	80.2	4.81	5.38	0.00
	random	76.23	-1.17	1.78	28.56×	3.65×	80.2	80.1	4.81	5.17	0.00
90%	magnitude_l1	70.24	-7.16	0.95	53.71×	14.52×	90.1	96.4	2.36	2.66	0.00
	magnitude_l2	71.41	-5.99	0.93	54.84×	14.67×	90.1	96.3	2.38	2.69	0.00
	bn_scale	68.81	-8.59	0.96	53.13×	14.09×	90.2	96.1	2.36	2.78	0.00
	fpgm	70.94	-6.46	0.94	54.27×	14.13×	90.2	96.3	2.35	2.96	0.00
	taylor	74.30	-3.10	1.52	33.54×	5.41×	90.1	88.1	2.50	2.96	0.00
	obdc	75.17	-2.23	1.36	37.37×	6.03×	90.2	89.2	2.46	2.92	0.00
	random	73.69	-3.71	1.34	38.05×	6.30×	90.1	90.6	2.48	3.02	0.00